PROJECT REPORT

On

“Random Password Generator”

[**Experiential Learning**]

Submitted By:

Bhumika Lekram Dandare [CM25012]

Guided By:

Mrs. Mayuri Getme



DEPARTMENT OF CSE (AI &ML)

S. B. JAIN INSTITUTE OF TECHNOLOGY MANAGEMENT AND RESEARCH, NAGPUR.

(An Autonomous Institute, Affiliated to RTMNU, Nagpur)

2026-2027

© S.B.J.I.T.M.R Nagpur

INDEX

INDEX		1
CHAPTER 1	INTRODUCTION	2-4
CHAPTER 2	METHODOLOGY	5-7
CHAPTER 3	TOOLS/PLATFORMS	8
CHAPTER 4	DESIGN & IMPLEMENTATION	9-12
	4.1 ALGORITHM	
	4.2 FLOWCHART	
	4.3 SOURCE CODE	
CHAPTER 5	RESULT & DISCUSSION	13-14
	5.1 OUTPUT	
	5.2 DISCUSSION	
	5.3 APPLICATION	
CHAPTER 6	CONCLUSION	15
	REFERENCES	16

CHAPTER 1

INTRODUCTION

1.1 Background Information

Passwords play an important role in protecting personal information and online accounts. A strong password helps improve account security, while weak or repeated passwords may be easier to guess. Creating different and complex passwords for multiple accounts can be difficult and time-consuming for users.

To address this problem, the Random Password Generator project has been developed using Python. It provides a simple way to generate passwords using a combination of uppercase letters, lowercase letters, numbers, and special characters. This project demonstrates how programming can be used to solve a practical, real-world problem.

1.2 Introduction to the Project

The Random Password Generator is a Python-based desktop application designed to generate passwords according to user requirements. The application allows users to customize password length and select the types of characters to include in the generated password.

The project uses Python modules such as string and secrets. The string module provides different character sets, while the secrets module supports secure random character selection. The application also demonstrates important programming concepts such as classes, functions, loops, conditional statements, and exception handling.

The main aim of this project is to make password generation simple, convenient, and customizable. It also provides practical experience in developing, testing, and maintaining a Python application.

1.3 Features of the Project

The major features of the Random Password Generator are:

1. Customizable Password Length: Users can specify the required length of the password.

CHAPTER 1

2. Character Selection: Users can select uppercase letters, lowercase letters, numbers, and special symbols.
3. Password Presets: Easy, Moderate, and Strong options help users choose predefined generation settings.
4. Password Strength Checker: Evaluates password strength based on criteria such as length and character variety.
5. Multiple Password Generation: Allows users to generate multiple passwords in one execution.
6. Input Validation: Checks user inputs and handles invalid selections.
7. File Handling: Provides an option to save generated passwords in a text file.
8. Object-Oriented Programming: Uses a class and methods to organize the program logic.

1.4 Objectives of the Project

The main objectives of this project are:

- To develop a password generator using Python.
- To generate passwords using secure random character selection.
- To provide flexibility in password length and character selection.
- To implement password strength checking.
- To understand Python modules, classes, loops, and conditional statements.
- To apply file handling and input validation.
- To gain practical experience in software development and testing.
- To maintain project code and documentation using GitHub.

1.5 Scope of the Project

The project focuses on generating customizable passwords through a local desktop application. It provides options for selecting character types, checking password strength, generating multiple passwords, and saving results, where these features are implemented in the application.
The source code and documentation can be maintained in a GitHub repository for version control and sharing. In the future, the application can be extended with additional customization options,

CHAPTER 1

an improved interface, and a browser-based version.

1.6 Significance of the Project

This project is useful for understanding how programming concepts can be applied to practical problems. It combines random character selection, user input, conditional logic, loops, object-oriented programming, and file handling in one application.

Developing this project also improves problem-solving skills, logical thinking, code organization, and testing practices. It provides a foundation for developing more advanced applications related to password management and digital security.

CHAPTER 2
METHODOLOGY

2.1 Introduction

The methodology describes the step-by-step process followed to develop the Random Password Generator project using Python. The development process started with identifying the problem and defining the requirements, followed by designing the program logic, implementing the code, testing the application, and maintaining the project files using GitHub.

2.2 Problem Identification

The first step was to identify the difficulty users face when creating strong and unique passwords manually. Simple passwords may be easier to guess, while creating different passwords for different accounts can be time-consuming. Therefore, a password generator was selected as a practical solution to simplify password creation.

2.3 Requirement Analysis

After identifying the problem, the requirements of the application were determined. The application was designed to generate passwords based on user preferences. The main requirements included selecting password length, choosing character types, generating passwords, and checking password strength. Additional requirements included input validation, generating multiple passwords, and saving passwords to a text file, wherever supported by the implementation.

2.4 System Design and Logic Development

After requirement analysis, the overall logic of the application was planned. The program accepts the required password settings from the user and prepares a collection of characters according to the selected options. It then selects characters randomly until the required password length is reached.

Conditional statements are used to identify the selected character types, while loops are used to repeat operations. Functions and class methods organize the different tasks into separate logical units.

CHAPTER 2

Table 2.1: Components and Rules of Password Generation

Component	Rule / Description
Password Length	As specified by the user.
Uppercase Letters	Include A–Z if selected.
Lowercase Letters	Include a–z if selected.
Numbers	Include 0–9 if selected.
Special Symbols	Include symbols if selected.
Character Set	Combination of all selected character types.
Password Generation	Randomly select characters until the required length is reached.
Password Strength	Check the generated password's strength, if implemented.
Multiple Passwords	Generate more than one password, if supported.
Save Password	Save the generated password to a text file, if implemented.
Input Validation	Check the entered length and selected options.
Final Output	Display the generated password to the user.

2.5 System Design and Logic Development

After requirement analysis, the overall logic of the application was planned. The program accepts the required password settings from the user and prepares a collection of characters according to the selected options. It then selects characters randomly until the required password length is reached.

Conditional statements are used to identify the selected character types, while loops are used to repeat operations. Functions and class methods organize the different tasks into separate logical units. Validation checks help the program handle invalid inputs and situations in which no character type has been selected.

2.6 Implementation

The planned logic was implemented using Python. The application was organized into suitable functions and class methods to make the code easier to understand and maintain. The password-

CHAPTER 2

generation component uses the selected character set to create passwords. Additional components, where implemented, evaluate password strength, generate multiple passwords, and save generated results in a text file.

The implementation stage involved writing the code, running the program, identifying errors, and making necessary corrections. The desktop interface was used to interact with the application and test its available functions.

2.7 Testing and Validation

Testing was performed to check whether the application worked according to the specified requirements. Different password lengths and character-selection combinations were tested. The strength-checking functionality, input validation, multiple password generation, and file-saving feature were also checked where implemented.

Errors identified during testing were corrected to improve the program's reliability. The testing process helped verify that the application generated passwords according to the selected settings and handled invalid inputs appropriately.

2.8 GitHub and Version Control

GitHub was used to maintain the project's source code and documentation in an online repository. The project files were uploaded to the repository, and commits were used to record changes made during development. This helped organize the project, maintain its development history, and make the source code accessible for review.

CHAPTER 3
TOOLS/PLATFORMS

1.1 SOFTWARE REQUIREMENT

S. No.	Software / Tool	Requirement
1	Operating System	Windows / Linux
2	Programming Language	Python 3.x
3	IDE / Code Editor	IDLE / VS Code / PyCharm
4	Python Modules	string, secrets, tkinter
5	Version Control	Git and GitHub
6	File Format	Python (.py) and text (.txt) files
7	User Interface	Tkinter GUI (Desktop Application)

1.1 SOFTWARE REQUIREMENTS

1.1.1 Operating System

Windows or Linux is required to run and develop the Random Password Generator application.

1.1.2 Programming Language

Python 3.x is used to develop the application and implement password-generation logic.

1.1.3 Code Editor

IDLE or Visual Studio Code can be used to write, edit, and execute the Python program.

1.1.4 Python Modules

The string module provides character sets, secrets supports secure random selection, and tkinter is used to create the graphical user interface.

1.1.5 Version Control

Git and GitHub are used to manage, store, and track changes to the project's source code.

1.1.6 File Support

Python files (.py) are used to store the program code, while text files (.txt) can be used to save generated passwords if the feature is implemented.

CHAPTER 4

DESIGN & IMPLEMENTATION

4.1 ALGORITHM

Step 1: Start the program.

Step 2: Ask the user to enter the required password length.

Step 3: Ask the user whether to include:

- Uppercase letters
- Lowercase letters
- Numbers
- Special symbols

Step 4: Create an empty character set.

Step 5: Add the selected character types to the character set.

Step 6: Check whether the character set is empty.

- If Yes, display "Please select at least one character type" and stop.
- If No, continue.

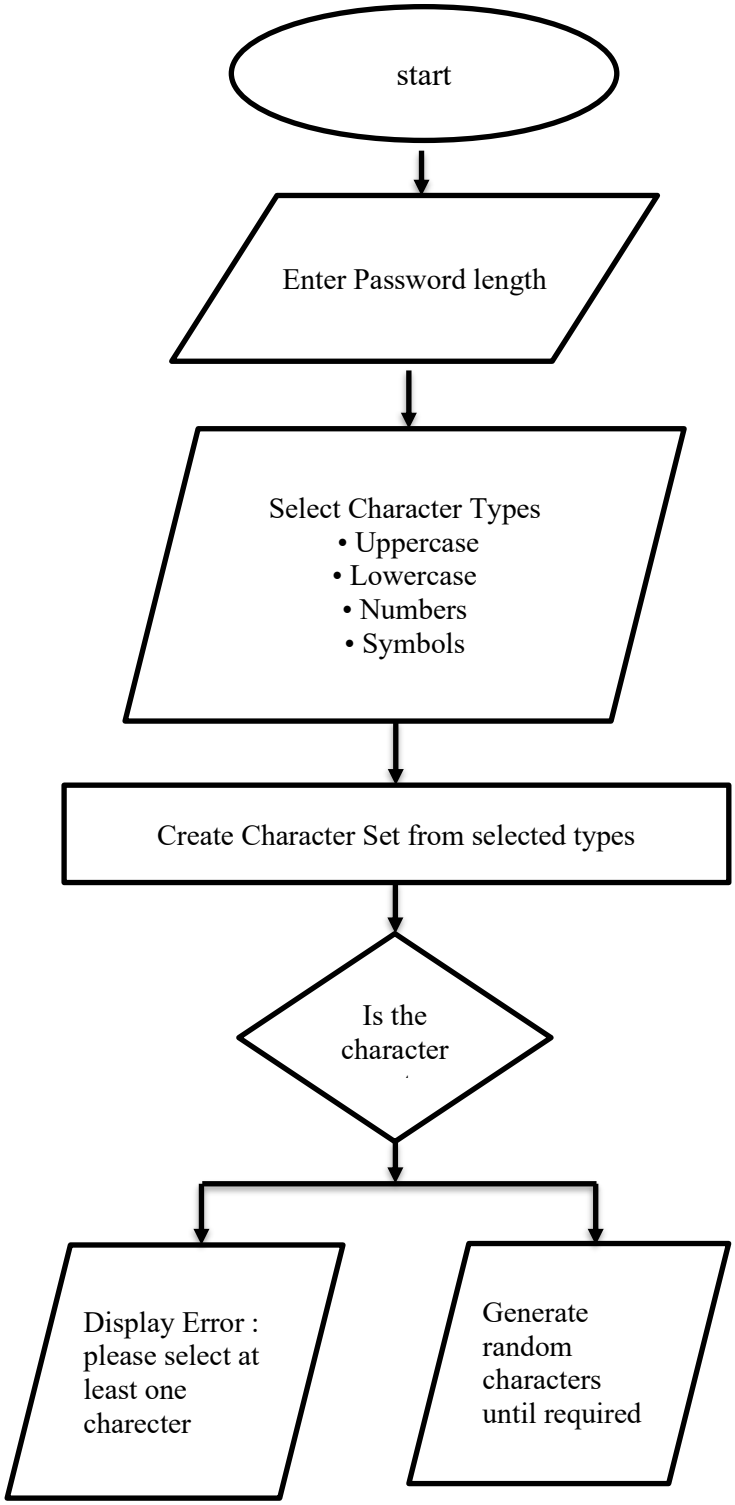
Step 7: Randomly select characters from the character set until the required password length is reached.

Step 8: Display the generated password.

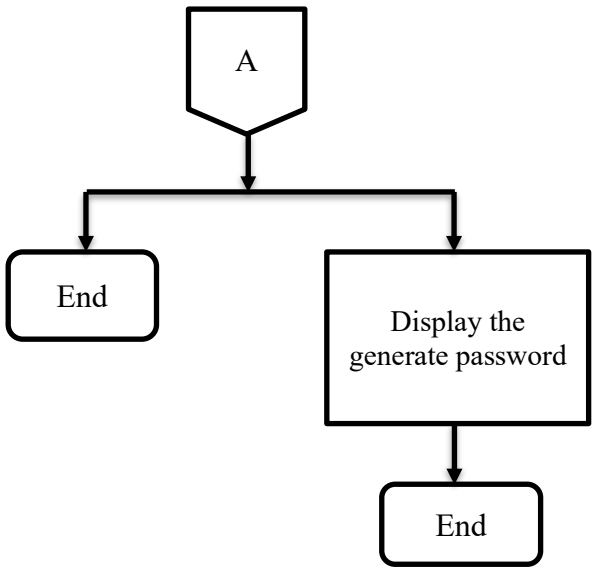
Step 9: End the program.

CHAPTER 4

4.2 FLOWCHART



CHAPTER 4



4.3 SOURCE CODE

```
import string
import secrets

print("===== Random Password Generator =====")

length = int(input("Enter password length: "))

uppercase = input("Include uppercase letters? (y/n): ").lower()
lowercase = input("Include lowercase letters? (y/n): ").lower()
numbers = input("Include numbers? (y/n): ").lower()
symbols = input("Include special symbols? (y/n): ").lower()

characters = ""

if uppercase == 'y':
    characters += string.ascii_uppercase
```

CHAPTER 4

```
if lowercase == 'y':
    characters += string.ascii_lowercase

if numbers == 'y':
    characters += string.digits

if symbols == 'y':
    characters += string.punctuation

if length <= 0:
    print("Please enter a valid password length.")

elif characters == "":
    print("Please select at least one character type.")

else:
    password = ""

    for i in range(length):
        password += secrets.choice(characters)

    print("\nGenerated Password:", password)

    print("Password Length:", len(password))

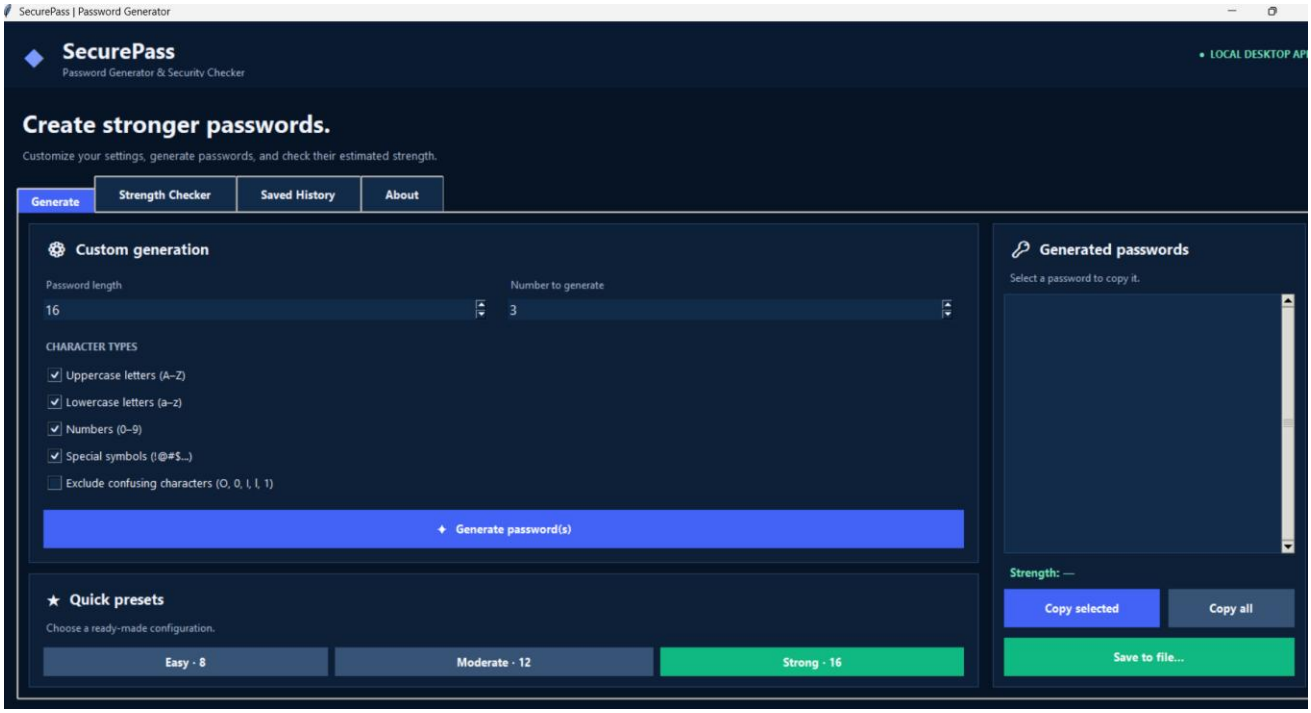
     print("===== Program Ended =====")
```

CHAPTER 5

RESULT & DISCUSSION

5.1 OUTPUT

5.1.1 Final Output



5.2 DISCUSSION

The Random Password Generator is a Python-based desktop application designed to generate random passwords according to user requirements. It allows users to select password length and character types, including uppercase letters, lowercase letters, numbers, and special symbols. The application uses Python modules such as string and secrets to generate passwords. It provides a simple interface and helps users create passwords conveniently.

CHAPTER 5

5.3 APPLICATION

1. Personal Accounts: Generating passwords for email and social media accounts.
2. Online Services: Creating passwords for websites and digital platforms.
3. Educational Purposes: Understanding Python programming and password generation.
4. Office Use: Helping users create passwords for work-related accounts.
5. Digital Security: Encouraging the use of unique and stronger passwords.
6. Wi-Fi Network Access**:** Generating strong passwords for personal or office Wi-Fi networks.
7. Software Development**:** Demonstrating the use of Python modules, functions, and graphical user interfaces.
8. Cybersecurity Awareness**:** Encouraging users to avoid simple and predictable passwords.
9. Temporary Accounts**:** Generating random passwords for test accounts used during software development.
10. Customized Password Creation: Creating passwords of different lengths using selected character types.
11. Project-Based Learning: Helping students gain practical experience in Python programming and application development.
12. Digital Privacy: Supporting better password practices to help protect personal information.

CHAPTER 6

CONCLUSION

The Random Password Generator using Python is a simple and useful desktop application that helps users create random passwords according to their requirements. It provides flexibility in selecting password length and character types, making password creation easier and more convenient.

The project demonstrates the practical implementation of Python programming concepts and modules such as string, secrets, and tkinter. It also provides an understanding of graphical user interface development, input validation, and application design.

Developing this project improved problem-solving skills, logical thinking, and practical programming knowledge. It also provided experience in organizing and maintaining source code using GitHub.

In conclusion, this project provides a basic solution for password generation and encourages users to follow better password practices. In the future, the application can be enhanced by adding features such as improved password strength analysis, clipboard copying, password history, and advanced security options.

The Random Password Generator reduces the effort required to create passwords manually and helps users avoid simple and predictable passwords. By combining different types of characters, it can generate passwords that are more difficult to guess. The project also demonstrates the importance of maintaining strong password practices to protect personal information and online accounts.

Furthermore, this project improved programming logic, problem-solving skills, and understanding of software development. It provided an opportunity to apply theoretical knowledge in a practical implementation.

In the future, this project can be enhanced by adding a graphical user interface (GUI), a password-strength checker, a copy-to-clipboard feature, and options to save generated passwords securely. It can also be improved by using Python's secrets module for cryptographically secure password generation.

In conclusion, the Random Password Generator is a simple, useful, and effective Python project that demonstrates the practical application of programming in cybersecurity. It helps users create stronger passwords and promotes awareness about the importance of password security in today's digital world.

REFERENCES

1. Python Software Foundation. Python 3 Documentation. Available at: https://docs.python.org/3/
2. Python Software Foundation. Secrets Module – Generate Secure Random Numbers. Available at: https://docs.python.org/3/library/secrets.html
3. Python Software Foundation. String Module – Common String Operations. Available at: https://docs.python.org/3/library/string.html
4. Python Software Foundation. Tkinter – Python Interface to Tcl/Tk. Available at: https://docs.python.org/3/library/tkinter.html
5. Python Software Foundation. Python Standard Library Reference. Available at: https://docs.python.org/3/library/
6. TutorialsPoint. Python Programming Tutorials. Available at: https://www.tutorialspoint.com/python/index.htm
7. GeeksforGeeks. Python Programming Language. Available at: https://www.geeksforgeeks.org/python/python-programming-language-tutorial/